Motif / Sequence Finder

Search every input sequence for a custom IUPAC nucleotide pattern, report all matches, and measure how sequestered each match is at every folding temperature.

Overview

Most of RSAS works on regions it locates for you: the RBS, the enclosing hairpin. The Motif / Sequence Finder is the opposite. You hand it a linear sequence pattern, and it finds every place that pattern occurs in each sequence, then runs the same structural-context measurements RSAS uses everywhere else against each hit.

The point is not just to locate a string. Any text editor can do that. The point is to ask a structural question about it: at each temperature you fold, how paired (sequestered) is this motif, and does that change as the temperature rises? That is the thermometer signal, applied to a region you defined instead of one RSAS picked.

The pattern is written in IUPAC nucleotide codes, so you can search for a single exact sequence (`GGAGG`) or a degenerate family (`NNUANN`, `RGGRG`) in one pass. Matching is done in RNA space: T is normalized to U before searching, so a DNA-style pattern still works.

When to use it

Reach for it when the region you care about is defined by sequence, not by the structural landmarks RSAS finds automatically.

- **Screening Shine-Dalgarno variants.** Search `AGGAGG` or a degenerate version like `RGGRGG` across a panel of leaders to see which constructs sequester their ribosome binding site when cold and release it when warm.
- **fourU and other known thermometer motifs.** The fourU class melts a short stretch that pairs the start-codon region. Give the finder the motif you expect and read its paired% across temperatures directly, without relying on the automatic RBS locator.
- **Any conserved element you already know.** A regulatory hairpin loop sequence, an anti-Shine-Dalgarno complement, a protein binding site. If you can write it as an IUPAC pattern, the finder will measure its sequestering at each hit.

If instead you want to find a *structural* motif (a helix plus a loop, not a flat sequence), that is the RNArobo Search page, not this tool.

Step by step

1. Open **Settings**, then the **Motif Finder** dialog.
2. Turn on **Enable motif search during analysis**. The pattern field and Validate button become active.
3. Type a pattern into **Motif Pattern**, for example `AGGAGG`, `UAUAAUGU`, or `NNUANN`.
4. Click **Validate** to check the pattern and preview how it expands. This is optional but catches typos before a full run.
5. Click **Save**.
6. Make sure the motif columns are turned on in **Output Column Settings** so the results appear in your exports.
7. Run your analysis as usual. Every sequence is searched, and the motif columns are filled in for each hit.

If you enable the finder but leave the pattern blank, Save warns you and does nothing until you either enter a pattern or disable the finder.

Options in detail

Enable toggle

`Enable motif search during analysis` is the master switch. When off, the pattern field and Validate button are disabled and no motif search runs. The setting is stored as `motif_search_enabled` in your saved settings, so it persists between sessions.

Pattern field

A single text field for one IUPAC pattern. It is searched against every sequence in the run. The pattern is upper-cased and trimmed before use, and the placeholder text suggests the format (`e.g. AGGAGG, UAUAAUGU, NNUANN`). The value is saved as `motif_pattern`.

IUPAC code table

Every character in the pattern is one of the standard four bases or a degenerate code. T is accepted and treated as U. The full set:

Code	Expands to	Meaning
`A`	A	Adenine
`C`	C	Cytosine
`G`	G	Guanine
`U` (or `T`)	U	Uracil (T normalized to U)

Code	Expands to	Meaning
`R`	A, G	puRine
`Y`	C, U	pYrimidine
`S`	G, C	Strong (3 H-bonds)
`W`	A, U	Weak (2 H-bonds)
`K`	G, U	Keto
`M`	A, C	aMino
`B`	C, G, U	not A
`D`	A, G, U	not C
`H`	A, C, U	not G
`V`	A, C, G	not U
`N`	A, C, G, U	aNy base

Internally each code is turned into a regular-expression character class (for example `R` becomes `[AG]` and `N` becomes `[ACGU]`), and the per-character pieces are concatenated into one pattern that is matched against each sequence.

Validate behavior

Click **Validate** to check the pattern without running an analysis. It applies these rules in order:

- **Empty.** Shows `Enter a motif pattern` in red and fails.
- **Invalid characters.** Anything outside `ACGURYWSKMBDHVN` is rejected, listing the offending characters in red.
- **Shorter than 3 nt.** Shows `Motif should be at least 3 nt long` in amber. This is a warning, not a failure: validation still passes, because a 1 to 2 nt pattern matches almost everywhere and is rarely what you want. You can save it anyway.
- **Longer than 30 nt.** Shows `Motif should be 30 nt or shorter` in red and fails. This is a hard cap.
- **Valid.** Shows the length and, for a degenerate pattern, a preview of the expanded regex (for example `Valid degenerate pattern (6 nt) -> [AG]GG[AG]GG`). An exact pattern reports `Valid exact pattern`.

Save runs the same validation: if the finder is enabled and the pattern does not pass, Save refuses.

Overlapping matches (allow_overlap)

Matching reports **all overlapping occurrences** by default. After a match, the search resumes one position past the match start (`pos = m.start() + 1`), not past the match end, so a pattern that repeats with overlap is found at every offset. For example `AA` in `AAAA` returns three hits (positions 0, 1, 2), not

two. This is the `allow_overlap=True` behavior built into `find_motif_occurrences`. With overlap disabled the search would resume past the match end and report only non-overlapping hits, but the analysis path uses the overlapping default.

What you get

When motif search is enabled, a Motif Finder column group is added to the output. The fields, per sequence:

- `Motif_Pattern` the IUPAC pattern you searched for.
- `Motif_Count` the number of matches found in that sequence.
- `Motif_Match_Seq` the matched subsequences. Semicolon-separated when there is more than one hit.
- `Motif_Match_Pos` the match positions, written as `start-end`, **0-based and half-open** (the `start` is included, the `end` is not, so a match at `4-9` covers positions 4, 5, 6, 7, 8).
- `Motif_Paired%_{T}C` the paired percentage of each match at each temperature, from the MFE dot-bracket structure.
- `Motif_Struct_{T}C` each match's dot-bracket substring at each temperature.
- `Motif_PF_Access_{T}C` the partition-function accessibility of each match, the mean unpaired probability over the match region. Present only when the partition function is enabled.
- `Motif_Paired_Diff_{hi}-{base}` the change in motif paired% from the base (first) temperature to the highest temperature, in percentage points.
- `Motif_PF_Diff_{hi}-{base}` the same difference for PF accessibility.
- `Motif_Paired_Diff_{mid}-{base}` / `Motif_PF_Diff_{mid}-{base}` the same differences for the middle temperature, present only when you fold at three or more temperatures.

When a sequence has several matches, every per-match field is semicolon-joined in the CSV in hit order, so `Motif_Match_Seq`, `Motif_Match_Pos`, the paired%, and the diffs all line up position by position.

In the **Excel** export, hits also get their own **Motif Matches** tab, one row per hit, so you do not have to split the semicolon-joined cells yourself. That tab is written only when there is at least one hit across the run.

For the complete column reference, see ../output-columns.md.

How it works

The work is done by `RnaThermofinder/utils/motif_finder.py`.

1. **Find.** `find_motif_occurrences(sequence, motif)` upper-cases the sequence and replaces T with U, converts the IUPAC pattern to a regex via `iupac_to_regex`, and scans for every (overlapping) match. Each hit is a dict with `start`, `end` (0-based, half-open), and `matched_seq`.
2. **Measure paired%.** For each hit and each temperature, `calc_motif_paired_percent(structure, start, end)` counts the paired positions (`(` or `)`) in the motif's slice of the MFE dot-bracket and divides by the motif length. It only runs when the structure exists and its length matches the sequence; otherwise the value is left empty for that temperature.
3. **Extract structure.** `calc_motif_dot_struct` pulls the dot-bracket substring for the motif region so you can see the local fold.
4. **Measure accessibility (optional).** When partition-function results are present, `calc_motif_pf_accessibility(unpaired_probs, start, end)` averages the per-position unpaired probability over the motif region. If the run used a windowed PF, the motif coordinates are shifted by the window offset first, and hits that fall outside the computed window are left empty.
5. **Temperature diffs.** `analyze_motif_sequestering` computes the high-minus-base and (for 3+ temperatures) middle-minus-base differences for both paired% and PF accessibility, per hit.

`analyze_motif_sequestering` ties these together for one sequence and returns the per-hit detail, the `motif_count`, the pattern, a `best_hit` (the most-paired match at the base temperature), and a flat summary used to fill the CSV columns.

For background on how the structures and unpaired probabilities themselves are computed, see ../methods.md.

Worked example

Search `GGAGG` (an exact 5 nt Shine-Dalgarno core) in a short sequence:

```
sequence: AUGGAGGUACGGAGGUU
position: 0         1
          0123456789012345 6
```

The pattern `GGAGG` is exact, so the regex is just `GGAGG`. Scanning with overlap, it matches at two places:

- Hit 1: `start = 2`, `end = 7`, `matched_seq = GGAGG`, reported as position `2-7`.
- Hit 2: `start = 10`, `end = 15`, `matched_seq = GGAGG`, reported as position `10-15`.

So `Motif_Count` is `2` and `Motif_Match_Pos` is `2-7;10-15`. If at 25 C the MFE structure over positions 2 to 7 is `(((((` (all paired) and over 10 to 15 is `..(((` (3 of 5 paired), then `Motif_Paired%_25C` reads `100.00;60.00`. Fold a second temperature where the first hit melts to `(....`, and `Motif_Paired_Diff_{hi}-25` shows a negative swing for that hit, the fingerprint you are looking for.

(The `GGAGG` matches here do not overlap. Overlap matters for repeats like `GG` inside `GGGG`, where every offset is reported.)

Tips

- **Watch the temperature-diff columns.** A single paired% at one temperature tells you little. The signal is the *change*: a motif that is highly paired when cold and much less paired when warm. A large negative `Motif_Paired_Diff` (or `Motif_PF_Diff`) is the thermometer fingerprint, the same logic RSAS applies to the RBS, now scoped to your motif.
- **Use PF accessibility for the honest answer.** Paired% comes from one MFE structure. When a leader has several competing folds, turn on the partition function and read `Motif_PF_Access` instead, which averages over the whole ensemble.
- **Validate before a long run.** A quick Validate click confirms the pattern expands the way you intended, especially for degenerate codes, before you commit to folding thousands of sequences.
- **Keep degenerate patterns specific.** A pattern full of `N`s matches almost everywhere and floods `Motif_Count`. Anchor it with fixed bases where you can.

Limitations and gotchas

- **Sequence patterns only.** This finds linear motifs. It cannot describe a helix or a loop. For structural motifs, use the RNArobo Search page.
- **Short patterns match everywhere.** Patterns under 3 nt trigger a warning for a reason. They produce huge hit counts and rarely carry a meaningful structural signal.
- **30 nt hard cap.** Patterns longer than 30 nt are rejected at validation. The tool is built for short regulatory elements, not whole-region search.
- **Overlapping hits are the default.** A repetitive pattern can report many overlapping matches at consecutive offsets. If you expected only non-overlapping hits, that is why the count is higher than you guessed.
- **Paired% needs a length-matched structure.** If the MFE structure for a temperature is missing or its length does not match the sequence, paired% and the motif dot-bracket are left empty for that temperature rather than guessed.
- **Windowed PF can drop hits.** When PF runs in a window, motif hits outside that window have no accessibility value. The MFE paired% columns are unaffected.
- **Hits are real positions, not best-only.** Every match is measured and reported. The internal `best_hit` (most paired at the base temperature) is a convenience, not a filter; your columns contain all hits.

Troubleshooting

- **No motif columns in the output.** Confirm the finder is enabled in the Motif Finder dialog *and* the motif columns are turned on in Output Column Settings. Both are required.
- `Not Found` / `N/A` **in every motif cell.** The pattern did not match that sequence. Check it against your data, remember T is treated as U, and verify degenerate codes expand the way you meant with Validate.
- **Save does nothing.** If the finder is enabled, Save runs validation first. An empty pattern pops a warning; an invalid or over-length pattern shows the reason in the dialog and Save refuses until you fix it.
- `Invalid characters` **on Validate.** The pattern contains a letter outside `ACGURYWSKMBDHVN`. Only the IUPAC codes in the table above are allowed.
- **More hits than expected.** Overlapping matches are reported by default, so repetitive patterns count every offset.
- **No Motif Matches tab in Excel.** That tab is written only when at least one hit exists across the whole run. No hits anywhere means no tab.